

ECOTRACK TEAM EXECUTION PLAN

Member 1: Authorization and Integration

A detailed implementation, ownership, testing, and integration plan for CASL authorization, tenant protection, shared composition files, and cross-organization security.

Primary owner	Team lead / Member 1
Primary branch	feature/authorization-integration
Base branch	integration/organization-onboarding
Milestone	Organization onboarding and Super Admin review
Prepared	10 August 2026
Status	Implementation-ready

Contents

1. Mission and success definition
2. Mandatory sources of truth
3. Branch and ownership rules
4. Dependencies and baseline checks
5. Authorization architecture
6. Actions, subjects, and ability types
7. Ability factory rules
8. Middleware and request context
9. Route authorization contracts
10. Shared backend integration
11. Shared frontend integration
12. Testing strategy
13. Handoffs and merge order
14. Definition of done
15. Reference material

1. Mission and success definition

Member 1 owns the authorization foundation and the final integration of the team's isolated feature modules. Authentication is already complete: Supabase verifies identity, the backend synchronizes the EcoTrack profile through Prisma, the web sends bearer tokens, role-aware login works, and the existing Super Admin check is proven. This work must extend that foundation without replacing it.

Core outcome: every organization application, Super Admin review, notification, and future organization-scoped request is allowed or denied by backend authorization built from trusted profile and membership data. Frontend visibility is never treated as security.

Deliverables

- Install and configure `@casl/ability` and `@casl/prisma`.
- Define stable application actions and subjects.
- Create typed ability rules from platform role and organization membership.
- Create ability-building and authorization middleware.
- Create tenant/membership middleware for organization-scoped operations.
- Extend Express request typing without weakening existing authentication types.
- Write negative and cross-organization tests.
- Integrate Member 2 and Member 3 routers into `backend/src/app.ts`.
- Integrate Member 2 and Member 3 pages into the real authenticated web flow.
- Run complete integration checks and protect shared files during merge.

2. Mandatory sources of truth

Read these before implementation and again before approving team pull requests:

- `docs/IMPORTANT/EcoTrack_Backend_Module_Structure_and_Heavy_Service_Reminder.txt`
- `docs/IMPORTANT/EcoTrack_Complete_System_Flow_Source_of_Truth_v1.txt`
- `docs/IMPORTANT/EcoTrack_Technical_Decision_Notes.txt`
- `docs/IMPORTANT/EcoTrack_Sri_Lanka_Service_Area_Boundaries_Guide.md`
- `database_docs/EcoTrack_Database_v2_Finalization_and_Codex_Instructions.txt`
- `database_docs/EcoTrack_ERD_v2_Final.dbml`
- `backend/prisma/schema.prisma` and all applied migrations

Scope boundary: do not implement incidents, latitude/longitude entry, point-to-GN matching, organization incident dashboards, or maps in this milestone. GN boundaries are used only for organization service-area selection and review.

3. Branch and ownership rules

Branch

```
git fetch origin
git switch integration/organization-onboarding
git pull --ff-only origin integration/organization-onboarding
git switch -c feature/authorization-integration
```

Owned paths

```
backend/src/authorization/**
backend/src/middleware/tenant.middleware.ts
backend/src/middleware/ability.middleware.ts
backend/src/middleware/authorize.middleware.ts
backend/src/types/express.d.ts (coordinated extension only)
backend/src/tests/authorization*.test.ts
backend/src/app.ts (integration owner only)
web/src/App.tsx (integration owner only)
```

Non-negotiable collaboration rules

- Do not delete or silently rewrite Member 2 or Member 3 feature files.
- Do not restore/apply the existing stash.
- Do not edit Member 2's migration files.
- Do not allow feature members to push directly to the integration branch.
- Keep shared-file wiring in a separate, reviewable commit.
- Use comments to explain authorization intent and security constraints, not authorship.

4. Dependencies and baseline checks

Install CASL from the backend directory. This is a shared package change owned by Member 1:

```
cd C:\campus\projects\Eco-Track\backend
npm install @casl/ability @casl/prisma
npm run typecheck
npm test
npm run build
```

Commit `package.json` and `package-lock.json` together. Never edit the lock file manually. The current `@casl/prisma` package supports modern Prisma clients and is installed together with `@casl/ability`.

Do not: remove the existing `requireActiveSuperAdmin` middleware until equivalent CASL routes are implemented, tested, and reviewed. Migration to CASL should be incremental, never a security gap.

5. Authorization architecture

Supabase Auth

-> proves identity and returns a verified user

Prisma repositories

-> load profile, platform role, organization membership and status

CASL ability factory

-> converts trusted context into permitted actions and subjects

Express middleware

-> enforces the ability before controllers

Service and repository

-> revalidate business invariants and scope database writes

Required request order

Route type	Required middleware order	Reason
Citizen creates organization application	authenticate -> ability -> authorize Create -> controller	No organization tenant exists yet; identity/profile is enough.
Citizen reads own application	authenticate -> ability -> authorize ReadOwn -> controller	Repository must also filter by authenticated profile ID.
Super Admin pending/review route	authenticate -> ability -> authorize Review/Approve/Decline -> controller	Platform-scoped authorization; no organization membership is required.
Future organization-owned route	authenticate -> tenant -> ability -> authorize -> controller	Membership and organization equality must be verified before access.
User notifications	authenticate -> ability -> authorize ReadOwn -> controller	Repository filters by authenticated recipient ID.

6. Actions, subjects, and ability types

Recommended file structure

```
backend/src/authorization/  
  actions.ts  
  subjects.ts  
  ability.types.ts  
  ability.factory.ts  
  permission.constants.ts
```

Actions

Use stable application vocabulary rather than route names. A suitable starting set is:

```
export const Actions = {
  Create: "create",
  Read: "read",
  ReadOwn: "readOwn",
  Update: "update",
  Review: "review",
  Approve: "approve",
  Decline: "decline",
  ManageMembership: "manageMembership",
} as const;
```

Subjects

```
export const Subjects = {
  OrganizationApplication: "OrganizationApplication",
  Organization: "Organization",
  OrganizationServiceArea: "OrganizationServiceArea",
  OrganizationMembership: "OrganizationMembership",
  Notification: "Notification",
} as const;
```

Important: do not use `manage all` as a shortcut for Super Admin. Project rules state that Super Admin handles platform governance and organization review, not ordinary organization cleanup work.

7. Ability factory rules

Trusted inputs

The ability factory may use only data loaded by backend authentication/repositories:

- Authenticated EcoTrack profile ID
- Platform role and account status
- Active organization memberships
- Membership role and membership status
- Verified active organization/tenant context when present

Initial policy matrix

Principal	Allowed	Explicitly not allowed
Active USER	Create an application; read own application; read own notifications	Review other applications; approve/decline; create own ORG_ADMIN role

Principal	Allowed	Explicitly not allowed
Active SUPER_ADMIN	List/review pending applications; approve/decline; view verification/service-area details; platform governance	Act as an organization admin without a real membership; ordinary tenant cleanup work
Active ORG_ADMIN membership	Future organization-scoped administration for that organization only	Access another organization by changing an ID; platform Super Admin review
Suspended/archived profile	Nothing protected	All protected operations
Suspended/left/removed membership	No organization-scoped operations	All tenant actions using that membership

Defense in depth

CASL answers whether an action is generally allowed. Repositories/services still scope queries using trusted IDs. For example, “read own application” must query using both application ID and `requestedByUserId = request.authentication.profile.id`. Never load by application ID and rely only on a frontend comparison.

8. Middleware and request context

Ability middleware

- Requires `request.authentication`.
- Loads any additional memberships needed for the request.
- Builds the ability through the factory.
- Attaches the typed ability to the request.
- Forwards unexpected errors to central error middleware.

Authorize middleware

- Accepts a required action and subject.
- Calls the request ability.
- Returns a consistent `403 FORBIDDEN` `ApplicationError` when denied.
- Contains no Prisma queries and no response-shaping controller logic.

Tenant middleware

Tenant middleware is required only when a route is truly organization-scoped. It must:

1. Read the organization ID from the route or another trusted route context.
2. Validate ID format.
3. Query a membership using authenticated profile ID plus organization ID.
4. Require active membership and a usable organization status.
5. Attach organization and membership context to the request.
6. Reject direct-ID attacks with 403 without revealing private tenant data.

Express type augmentation

Extend the existing declaration rather than replace it:

```
declare global {
  namespace Express {
    interface Request {
      authentication: AuthenticationContext;
      ability?: AppAbility;
      tenant?: OrganizationTenantContext;
    }
  }
}
```

Type declarations describe properties; middleware creates the actual runtime values.

9. Route authorization contracts

Endpoint	Permission	Additional backend scope
POST /api/v1/organization-applications	Create OrganizationApplication	Requester ID comes from auth context; selected areas must exist and be active references.
GET /api/v1/organization-applications/me	ReadOwn OrganizationApplication	Filter by authenticated profile ID.
GET /api/v1/super-admin/organization-applications	Review OrganizationApplication	SUPER_ADMIN and active account.
GET /api/v1/super-admin/organization-applications/:id	Review OrganizationApplication	Return review-safe details.
POST .../:id/approve	Approve OrganizationApplication	Conditional pending-state transaction.
POST .../:id/decline	Decline OrganizationApplication	Conditional pending-state transaction and required review notes policy.
GET /api/v1/notifications	ReadOwn Notification	Recipient filter from auth context.

10. Shared backend integration

`backend/src/app.ts` remains a composition root. Preserve this order:


```
security middleware
cors
json parser
health
versioned routers
notFoundMiddleware
errorMiddleware
```

Member 2 and Member 3 export router factories and dependency objects. Member 1 mounts them. Do not import repositories directly into `app.ts` ; routers receive dependencies or import their module dependency object.

Integration review checklist

- ☐ No duplicate `/api/v1` prefix.
- ☐ Authentication runs before ability/tenant middleware.
- ☐ Authorization runs before the controller.
- ☐ Controller delegates business rules to the service.
- ☐ Service delegates database operations to repository/transaction helpers.
- ☐ `notFound` and error middleware remain last.
- ☐ No route trusts body-supplied actor IDs or roles.

11. Shared frontend integration

The current web authentication state must remain intact. Member 1 integrates exported pages into the signed-in experience and routes users by capability:

```
Authenticated USER
  -> organization application/status UI

Authenticated SUPER_ADMIN
  -> pending organization review UI

Unauthorized frontend state
  -> hide action for UX
  -> backend still rejects direct calls
```

Use the existing `apiClient.ts` so bearer tokens and backend errors remain consistent. Do not put CASL alone in the browser and treat it as enforcement. Browser authorization is presentation only.

12. Testing strategy

Authorization unit tests

- Active USER can create application.
- USER cannot review/approve/decline.
- Active SUPER_ADMIN can review/approve/decline.
- Suspended SUPER_ADMIN cannot perform review.
- ORG_ADMIN rules contain organization conditions.
- Suspended/left membership produces no tenant capability.

Middleware tests

- Missing authentication produces 401.
- Missing ability context fails safely.
- Denied action produces stable 403 JSON.
- Tenant middleware rejects membership from another organization.
- Tenant middleware rejects suspended organization/membership.

Cross-organization tests

Cross-organization tests are mandatory. A successful same-organization test does not prove tenant security.

- Organization A admin cannot read or update Organization B private records by changing an ID.
- A membership for A cannot authorize a route for B.
- Frontend-supplied role and actor IDs are ignored.
- A normal user cannot call Super Admin endpoints directly even if the button is hidden.
- Review queries never leak documents/notes to unauthorized accounts.

Final commands

```
cd backend
npm run typecheck
npm test
npx tsx --test src/tests/authorization.integration.test.ts
npm run build
npx prisma validate
npx prisma migrate status
npm run db:check

cd ../web
npm run build
npm run lint

cd ..
git diff --check
git status --short
```

13. Handoffs and merge order

1. Publish action/subject names and middleware contract to Members 2 and 3 early.
2. Merge CASL dependency and base authorization types before feature route integration.
3. Review Member 2 schema/migration and organization application module.
4. Merge/apply Member 2 migration before Member 3 real Prisma review repository.
5. Review Member 3 transaction for conditional state transitions, membership creation, audit, and notification.
6. Mount routers in `app.ts` and pages in web integration.
7. Run all auth, authorization, organization, review, and web checks.

14. Definition of done

- ☐ CASL dependencies installed and lockfile committed.
- ☐ Typed actions, subjects, ability and middleware implemented.
- ☐ USER and SUPER_ADMIN policy matrix covered by tests.
- ☐ Tenant middleware has Organization A vs Organization B tests.
- ☐ Member 2 and Member 3 routes use agreed authorization middleware.
- ☐ Shared `app.ts` order remains correct.
- ☐ Web pages remain connected to real authentication and bearer API client.
- ☐ No secret, environment file, stash content, or unrelated edit is included.
- ☐ Backend tests/typecheck/build and web build/lint pass.
- ☐ Database validates and migration status is current.
- ☐ Pull requests include setup, API contracts, tests, and limitations.

15. Reference material

- Local source: `docs/IMPORTANT/EcoTrack_Backend_Module_Structure_and_Heavy_Service_Reminder.txt`
- Local source: `database_docs/EcoTrack_Database_v2_Finalization_and_Codex_Instructions.txt`
- CASL ability package: <https://www.npmjs.com/package/@casl/ability>
- CASL Prisma package: <https://www.npmjs.com/package/@casl/prisma>

EcoTrack team plan - Member 1. This document defines the current milestone only. Any change to role semantics, tenant ownership, schema responsibilities, or excluded incident/map scope requires a documented team decision before implementation.